

Student

I've been designing schemas for a **warehouse**. But my company keeps mentioning **data lakes** and **lakehouses**. What are the differences, and where does the star schema I just built actually live?

ChatGPT • AI Assistant

These three names describe the **three dominant storage-and-compute architectures** for analytics today. Your star schema can live in any of them - but each has different strengths, costs, and constraints. This chapter teaches:

1. A recap of the **data warehouse** and where it shines.
2. What a **data lake** is - schema-on-read on cheap object storage.
3. The **lakehouse** - the modern hybrid that combines both.
4. **Architecture** side-by-side.
5. **Trade-offs** across governance, performance, and flexibility.
6. **Cost dimensions** - storage vs compute vs query.
7. **Open table formats** - Delta, Iceberg, Hudi.
8. The **medallion architecture** - Bronze, Silver, Gold layers.
9. A **decision framework** for choosing the right architecture for your team.

**CONCEPT CARD**

The three patterns in one line each:

- **Data warehouse** = structured, governed, SQL-first, best BI performance, higher cost per TB.
- **Data lake** = raw files in cheap object storage, any data type, minimal governance, huge scale.
- **Lakehouse** = warehouse features (ACID, schema, SQL) *directly on* lake storage - one physical layer for BI and ML.

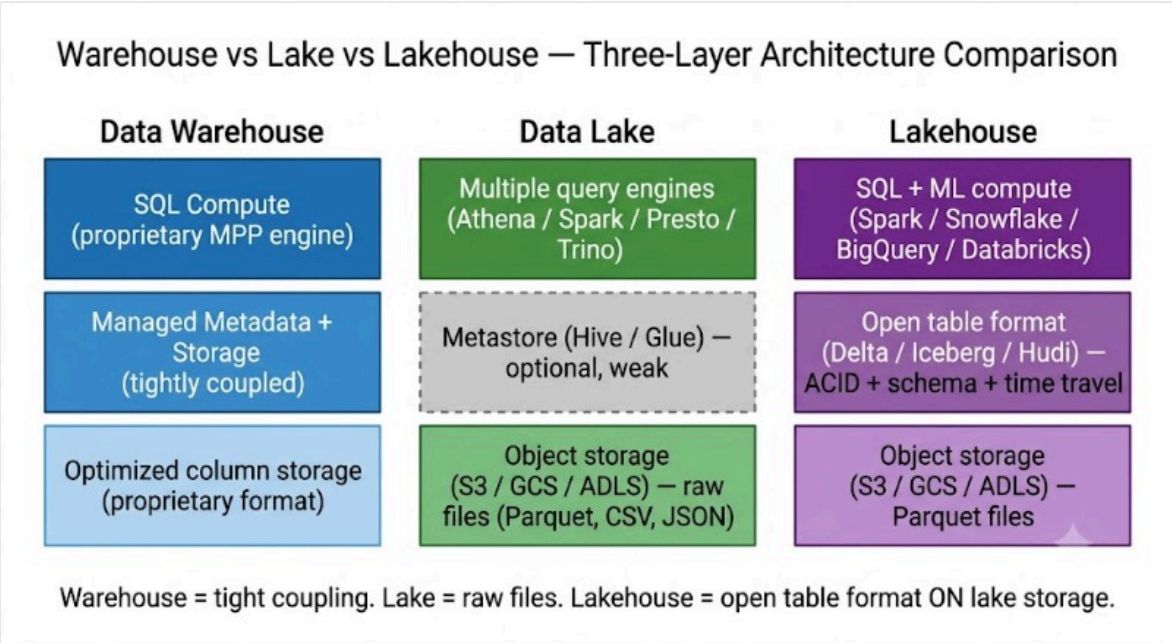
**SUMMARY CARD**

By the end of this chapter you'll be able to:

- Explain each architecture in one sentence.
- Draw the three-layer stack (storage / metadata / compute) for each.
- Recognize when each is the right choice.

- Talk fluently about Delta, Iceberg, Hudi, and the medallion pattern.
- Estimate cost trade-offs across storage, compute, and query.

[AI Prompt] Warm-up (paste into any AI chatbot): *"Explain the difference between a data warehouse, a data lake, and a lakehouse in one paragraph each. Include what kind of data each stores, how you query it, and one real product example for each."*



Warehouse vs Lake vs Lakehouse — Three-Layer Architecture Comparison

| Warehouse vs Lake vs Lakehouse — Head-to-Head |                           |                                   |                                  |
|-----------------------------------------------|---------------------------|-----------------------------------|----------------------------------|
| Aspect                                        | Warehouse                 | Data Lake                         | Lakehouse                        |
| Storage                                       | Managed, column-optimized | Raw files on object storage       | Parquet on object storage        |
| Schema                                        | Schema-on-write (strict)  | Schema-on-read (loose)            | Schema-on-write via table format |
| Data types                                    | Structured only           | Any (images, video, JSON, tables) | Any + tables via format          |
| ACID transactions                             | Yes                       | No                                | Yes (via Delta/Iceberg/Hudi)     |
| Storage cost                                  | Higher                    | Lowest                            | Lowest (same as lake)            |
|                                               |                           |                                   |                                  |

| Aspect            | Warehouse                     | Data Lake                | Lakehouse                       |
|-------------------|-------------------------------|--------------------------|---------------------------------|
| Query performance | Fastest for BI                | Slower on complex joins  | Nearly warehouse-level          |
| Governance        | Excellent                     | Weak (data swamp risk)   | Strong (via table format)       |
| ML / data science | Limited                       | Excellent                | Excellent                       |
| Example products  | Snowflake, BigQuery, Redshift | S3 + Athena, HDFS + Hive | Databricks, Snowflake + Iceberg |

sql – The same star-schema fact – three ways it can be created

```
-- =====
-- 1) WAREHOUSE (Snowflake / BigQuery / Redshift)
-- Managed column storage; proprietary format underneath
-- =====

CREATE TABLE analytics.fact_sales (
  sales_key    BIGINT,
  date_key     INT,
  customer_key BIGINT,
  product_key  BIGINT,
  net_amount   DECIMAL(12, 2)
);

-- =====
-- 2) DATA LAKE (raw Parquet on S3, queried by Athena / Spark)
-- Schema is defined only for querying, not for storage
-- =====

CREATE EXTERNAL TABLE fact_sales (
  sales_key    BIGINT,
  date_key     INT,
  customer_key BIGINT,
  product_key  BIGINT,
  net_amount   DECIMAL(12, 2)
)
STORED AS PARQUET
LOCATION 's3://my-lake/analytics/fact_sales/';

-- =====
```

```
-- 3) LAKEHOUSE (Delta / Iceberg on S3)
-- Parquet under the hood, PLUS ACID + schema evolution
-- =====
CREATE TABLE analytics.fact_sales (
  sales_key    BIGINT,
  date_key     INT,
  customer_key BIGINT,
  product_key  BIGINT,
  net_amount   DECIMAL(12, 2)
)
USING DELTA
LOCATION 's3://my-lakehouse/analytics/fact_sales/';

-- Same star schema. Three different homes. Different trade-offs.
```

## DATA WAREHOUSE — RECAP AND STRENGTHS

Student

Chapter 1 introduced the data warehouse. Give me the quick recap and its **superpowers**.

ChatGPT • AI Assistant

A **data warehouse** is a purpose-built system for analytical SQL over structured data. It uses **schema-on-write** (you define tables before loading), stores data in **column-optimized format**, runs **massively parallel** SQL, and offers strong **governance** - access control, lineage, auditability.

### CONCEPT CARD

Warehouse strengths (why it still dominates for BI):

1. **Predictable, fast SQL** - designed for concurrent analyst queries.
2. **Strong governance** - row/column security, audit logs, lineage.
3. **Battle-tested optimizer** - decades of tuning for star schemas.
4. **First-class BI-tool support** - Tableau, Power BI, Looker are native.
5. **Managed operations** - cloud vendors handle backups, upgrades, tuning.

## 6. Time-tested SLA and cost predictability - for pure SQL workloads.

Student

What's the flip side?

ChatGPT • AI Assistant

### EXAMPLE CARD

Warehouse limitations:

1. **Structured only** - no native image, video, or unstructured text handling.
2. **Schema rigidity** - evolving schemas requires migrations.
3. **Cost per TB** - storage priced higher than raw object storage.
4. **Data-science friction** - ML/notebook workflows fit awkwardly; often data gets copied out.
5. **Vendor lock-in risk** - proprietary formats and features.

### 🗨️ ANALOGY

A warehouse is like a **luxury hotel**. Everything is polished, service is fast, staff know your name - but you pay for that experience per night, and you can't bring your own furniture.

### ⚠️ WARNING

Modern warehouses (Snowflake, BigQuery) are borrowing lakehouse features. The line between them is blurring. Read current docs - the shape of what "warehouse" means keeps shifting.

### ❗ COMMON PITFALL / MISTAKE

**Incorrect Way:** Dismissing warehouses as legacy. They're still the fastest, most governed option for pure SQL analytics.

**Correct Way:** Use a warehouse when your workload is 95%+ SQL analytics on structured data and you value governance and simplicity over cost or flexibility.

### EXAM TIP CARD

Interview classic: *"When is a warehouse still the right choice?"* Winning answer: *"When workload is dominated by SQL analytics on structured data, governance and BI-tool integration matter, and the team doesn't need to handle images, audio, or ML-scale unstructured data. Snowflake, BigQuery, and Redshift are the mainstream picks."*

### 🔥 EXERCISE

List 3 warehouse products used in your industry. For each, note the pricing model (per-TB compute? per-query? per-cluster?) and one strength you know of.

[AI Prompt] Paste this into any AI chatbot: *"Compare Snowflake, BigQuery, and Redshift in a table. Include storage pricing model, compute pricing model, best-fit use case, and one gotcha for each."*

### 🔗 PRACTICE QUIZ

What kind of data is a data warehouse designed to handle?

**Answer: Structured, SQL-queryable data** - tables with defined schemas.

#### ▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

## DATA LAKE — SCHEMA-ON-READ AND OBJECT STORAGE

ChatGPT • AI Assistant

text – Typical data-lake folder layout on S3

```
s3://acme-lake/
├── raw/                                # untouched source dumps
│   ├── stripe/
│   │   ├── payments/
│   │   │   ├── 2026/07/01/*.json
│   │   │   └── 2026/07/02/*.json
│   │   └── salesforce/
│   │       ├── accounts/
│   │       │   └── 2026/07/*.csv
│   │   └── product_db/                # from operational Postgres
│   │       ├── orders/
│   │       │   └── 2026/07/*.parquet
│   └── curated/                       # cleaned, validated
```

```

├── sales/
│   └── year=2026/month=07/*.parquet
├── customers/
│   └── year=2026/month=07/*.parquet
└── analytics/                                # business-ready star schema
    ├── fact_sales/
    │   └── year=2026/month=07/*.parquet
    └── dim_customer/
        └── *.parquet

```

# Discipline: predictable folder structure = the closest thing  
# a data lake has to a schema. Enforce with orchestration.

sql – Data lake: schema-on-read with Athena / Trino / Spark SQL

```

-- Register a schema for the Parquet files sitting on S3.
-- The files exist regardless; this just tells the engine
-- how to interpret them.

```

```

CREATE EXTERNAL TABLE fact_sales (
    sales_key    BIGINT,
    date_key     INT,
    customer_key BIGINT,
    product_key  BIGINT,
    net_amount   DECIMAL(12, 2)
)
PARTITIONED BY (year INT, month INT)
STORED AS PARQUET
LOCATION 's3://acme-lake/analytics/fact_sales/'
TBLPROPERTIES ('parquet.compression' = 'SNAPPY');

```

```

-- Refresh partition metadata after new files land
MSCK REPAIR TABLE fact_sales;

```

```

-- Same star-schema query pattern as in a warehouse
SELECT date_key, SUM(net_amount) AS revenue
FROM   fact_sales
WHERE  year = 2026 AND month = 7
GROUP BY date_key
ORDER BY date_key;

```

What is a **data lake**?

ChatGPT • AI Assistant

A **data lake** is a store of **raw files** in cheap **object storage** (Amazon S3, Google Cloud Storage, Azure Data Lake Storage). It's called a *lake* because you can pour anything in - CSV, JSON, Parquet, images, video, PDFs - without deciding structure up front.

**Schema is applied when you read**, not when you write. Compute engines (Spark, Athena, Presto, Trino) interpret the files at query time.

#### CONCEPT CARD

Data lake defining features:

1. **Storage-compute separation** - files sit on S3; any engine can query them.
2. **Schema-on-read** - no strict schema when loading; each query interprets the files.
3. **Any data type** - tables, images, audio, JSON events, logs.
4. **Extremely cheap storage** - object storage is 5-10× cheaper per TB than a warehouse.
5. **Multiple engines** - SQL (Athena), Spark, ML frameworks all read the same files.
6. **Open file formats** - Parquet, ORC, JSON, CSV.

Student

What's the catch?

ChatGPT • AI Assistant

#### ⚠ WARNING

Data lakes come with a real risk - **the data swamp**. Without discipline:

1. Nobody knows what's in which file.
2. Schemas drift silently - one Parquet file has **amount**, the next has **net\_amount**.
3. No ACID transactions - concurrent writes corrupt data.
4. No **UPDATE** or **DELETE** on Parquet - you rewrite whole partitions.
5. Weak governance - access control at the file level, not row-level.

#### EXAMPLE CARD

Typical data-lake tech stack:

1. **Storage** - S3, GCS, or Azure Data Lake Storage (ADLS).
2. **File formats** - Parquet (columnar, compressed), ORC, JSON.
3. **Metastore** - AWS Glue Catalog or Hive Metastore (maps folder → schema).
4. **Query engines** - Amazon Athena, Google BigQuery (external), Apache Spark, Presto/Trino.
5. **Orchestration** - Airflow, dbt on Spark, Databricks jobs.

### 🔗 ANALOGY

A data lake is like a **giant self-storage warehouse**. Cheap by the square foot. Store anything - furniture, boxes, cars. But nobody labels their unit clearly, and if you don't have a good system, you can lose things forever.

### ⚠️ COMMON PITFALL / MISTAKE

**Incorrect Way:** Dumping every source's raw data into an unversioned S3 bucket "we'll organize it later." Six months later, the swamp is unusable.

**Correct Way:** Impose folder discipline:

`s3://lake/<domain>/<table>/<yyyy>/<mm>/<dd>/` . Register schemas in a metastore. Enforce partition conventions.

### EXAM TIP CARD

Interview classic: *"What is schema-on-read?"* Winning answer: *"Data is stored as raw files without enforcing a schema. Each query interprets the files using a schema chosen at read time. Trade-off: extreme flexibility for less integrity and slower queries."*

### 🔥 EXERCISE

Sketch a lake folder structure for a company: `s3://lake/sales/< ... >` , `s3://lake/events/< ... >` , `s3://lake/images/< ... >` . Include date-based partitions. Notice how folder layout is the closest thing to a "schema" in a lake.

[AI Prompt] Paste this into any AI chatbot: *"Explain data lake schema-on-read with a concrete example: raw Parquet files landed on S3, an Athena CREATE EXTERNAL TABLE, and a query. Highlight what happens when the file schema differs from the table definition."*

## THE LAKEHOUSE — BEST OF BOTH WORLDS

sql – Lakehouse in action – ACID, MERGE, and time travel

```
-- =====
-- Create a Delta / Iceberg lakehouse table
-- =====

CREATE TABLE analytics.fact_sales (
    sales_key      BIGINT,
    date_key       INT,
    customer_key   BIGINT,
    product_key    BIGINT,
    net_amount     DECIMAL(12, 2),
    loaded_ts      TIMESTAMP
)
USING DELTA                                -- or ICEBERG
PARTITIONED BY (date_key)
LOCATION 's3://acme-lakehouse/analytics/fact_sales/';

-- =====
-- MERGE (upsert) – impossible on a plain data lake
-- =====

MERGE INTO analytics.fact_sales AS tgt
USING staging.fact_sales_incoming AS src
    ON tgt.sales_key = src.sales_key
WHEN MATCHED THEN
    UPDATE SET
        net_amount = src.net_amount,
        loaded_ts  = CURRENT_TIMESTAMP
WHEN NOT MATCHED THEN
    INSERT (sales_key, date_key, customer_key, product_key,
            net_amount, loaded_ts)
    VALUES (src.sales_key, src.date_key, src.customer_key,
            src.product_key, src.net_amount,
            CURRENT_TIMESTAMP);

-- =====
-- Time travel – query the table as it was earlier
-- =====

-- Delta syntax:
SELECT COUNT(*)
FROM   analytics.fact_sales
VERSION AS OF 42;                                -- version number

SELECT COUNT(*)
FROM   analytics.fact_sales
```

```

TIMESTAMP AS OF '2026-07-01 09:00:00';           -- as-of point-in-time

-- =====
-- Schema evolution – add a column without a migration
-- =====

ALTER TABLE analytics.fact_sales
  ADD COLUMN promotion_key BIGINT;

-- =====
-- View the transaction log (Delta)
-- =====

DESCRIBE HISTORY analytics.fact_sales;

```

Student

What is a **lakehouse**?

ChatGPT • AI Assistant

A **lakehouse** takes lake storage (cheap object storage, Parquet files) and adds a **table format layer** on top that gives you **ACID transactions**, **schema enforcement**, **time travel**, and **fast SQL** - the features that used to only exist in warehouses. It's the current industry direction: one physical layer for BI, ML, and data engineering.

**CONCEPT CARD**

What lakehouses add on top of a data lake:

1. **ACID transactions** - atomic multi-row writes, consistent reads.
2. **Schema enforcement + evolution** - reject bad writes; add columns safely.
3. **Time travel** - query the table "as of yesterday" with one clause.
4. **UPDATE / DELETE / MERGE** - impossible in a plain lake; standard in a lakehouse.
5. **Statistics and indexes** - small-file compaction, Z-ordering, data skipping.
6. **Single source of truth** - same tables for BI dashboards, ML training, and streaming.

Student

How is this different from a warehouse?

**EXAMPLE CARD**

Key differences from a classic warehouse:

1. **Storage is open** - Parquet on S3/GCS/ADLS you control, not a proprietary format.
2. **Storage-compute separation** - swap engines (Spark, Snowflake, DuckDB) without moving data.
3. **Multi-modal** - the same platform handles tables AND images/audio/text for ML.
4. **Open-source table formats** - Delta, Iceberg, Hudi (Section 5-7) - vendor-neutral.
5. **Cheaper for large scale** - you pay object-storage prices for the actual bytes.

**🔗 ANALOGY**

The lakehouse is like a **modern furnished apartment** in a converted warehouse. You get the low rent and huge footprint of the warehouse building (data lake), plus the finishes, plumbing, and security of a proper home (warehouse features). Best of both.

**⚠️ WARNING**

Not every workload needs a lakehouse. Small structured-only workloads still run beautifully on a plain warehouse. Adding open table formats has real complexity - compaction jobs, catalog management, engine compatibility.

**❗ COMMON PITFALL / MISTAKE**

**Incorrect Way:** "Lakehouse" as a marketing sticker on any lake with a metastore. Without ACID transactions and schema enforcement, it's still a lake.

**Correct Way:** A real lakehouse uses an open table format (Delta, Iceberg, Hudi) that provides ACID, schema evolution, and time travel - verify these features exist before calling it a lakehouse.

**EXAMPLE CARD**

Common lakehouse platforms in 2026:

1. **Databricks** - Delta Lake native; the pattern's originator.
2. **Snowflake with Iceberg tables** - warehouse doubling as lakehouse via open format.
3. **BigQuery + BigLake** - Google's lakehouse story on Iceberg.
4. **Trino / Presto + Iceberg** - open-source combo, no vendor lock-in.
5. **Databricks + Iceberg / Delta UniForm** - cross-format interop.

**EXAM TIP CARD**

Interview classic: *"What's a lakehouse and how is it different from a lake?"* Winning answer: *"A lakehouse is a data lake plus an open table-format layer (Delta / Iceberg / Hudi) that adds ACID transactions, schema enforcement, time travel, and warehouse-class SQL performance - all on cheap object storage."*

### 🔥 EXERCISE

For any lakehouse platform, list its table format, its default file format, and one governance feature it adds over a plain lake. If you can, run a **DESCRIBE HISTORY** on a Delta or Iceberg table - that's the time-travel feature in action.

[AI Prompt] Paste this into any AI chatbot: *"Explain how a Delta Lake table on S3 achieves ACID transactions despite Parquet files not being editable. Include what the transaction log is and how MERGE INTO works under the hood."*

## ARCHITECTURE COMPARISON

Student

Draw me the **three architectures side by side**.

ChatGPT • AI Assistant

Every analytics stack has three layers: **storage** (bytes on disk), **metadata** (schema, table definitions, statistics), and **compute** (query engines). Warehouse, lake, and lakehouse differ in how tightly these layers are coupled and where each lives.

### CONCEPT CARD

Layer-by-layer breakdown:

1. **Data Warehouse** - all three layers **tightly coupled** inside one product. Snowflake owns the file format, the metadata, and the compute.
2. **Data Lake** - layers **loosely coupled**. Files live on S3, metadata in a Hive/Glue catalog (or nothing), compute is any engine that can read the files.
3. **Lakehouse** - layers **cleanly separated**. Files on S3 (Parquet), metadata in an open table format (Delta / Iceberg / Hudi transaction log), compute is any engine that supports the format.

Student

## Why does layer separation matter?

ChatGPT • AI Assistant

**EXAMPLE CARD**

Consequences of coupling vs separation:

1. **Coupled (warehouse)** - great performance, vendor lock-in, higher costs, no ML flexibility.
2. **Loose (lake)** - max flexibility, no ACID, weak governance, data-swamp risk.
3. **Separated (lakehouse)** - best of both - but requires discipline in choosing a table format and compaction strategy.

**🔗 ANALOGY**

Think of the three layers as **restaurant, kitchen, and pantry**:

- Warehouse - one restaurant that runs its own kitchen using its own private pantry. Consistent, premium.
- Lake - a farmer's market pantry, no kitchen. Anyone can cook - if they bring their own stove and know what they're doing.
- Lakehouse - the same farmer's market pantry, plus standard-issue kitchens anyone can rent. Same ingredients, many chefs, consistent output.

**⚠️ WARNING**

The clean picture above blurs in real life. Modern Snowflake supports Iceberg tables (lakehouse mode). Databricks Runtime can write to Delta (lakehouse) or query external tables (lake). Products increasingly cover multiple architectures at once.

**⚠️ COMMON PITFALL / MISTAKE**

**Incorrect Way:** Choosing based on marketing labels. "Cloud data warehouse" and "lakehouse platform" often overlap in features.

**Correct Way:** Look at the actual capabilities - does it use an open table format? Does it separate storage and compute? What formats does it read/write?

**EXAM TIP CARD**

Interview favorite: *"What are the three layers of an analytics stack?"* Winning answer: *"Storage (the bytes), metadata (schema and stats), and compute (query engines). Warehouses tightly couple all three; lakes loosely couple; lakehouses cleanly separate with an open metadata layer like Delta or Iceberg."*

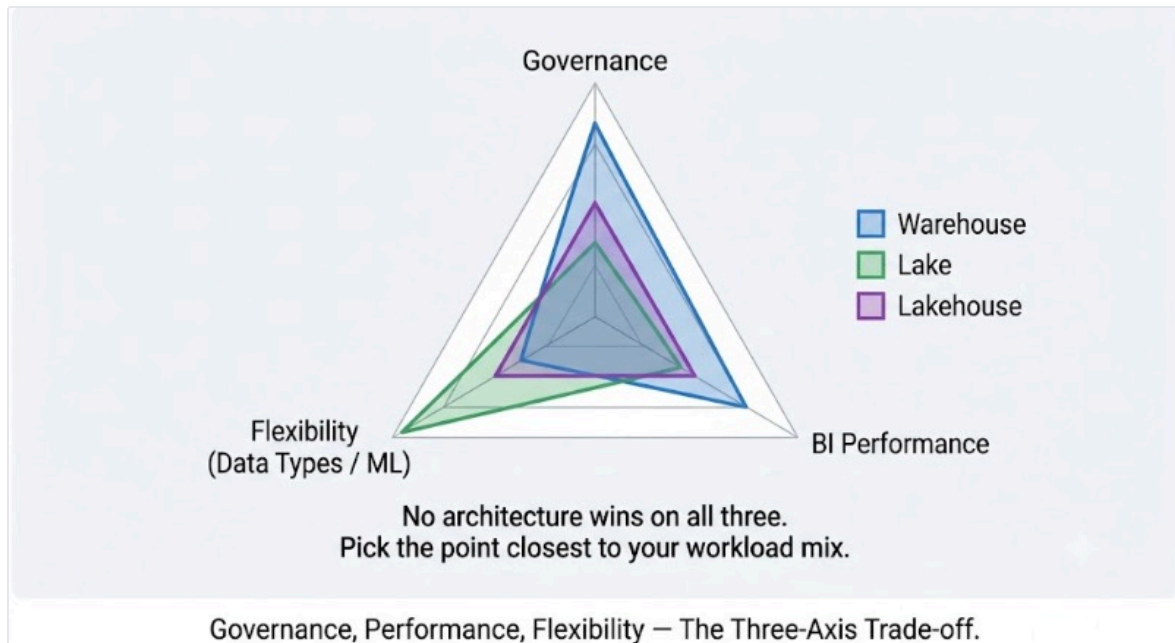
**EXERCISE**

For your team's current data platform, identify each layer: What's the storage? What's the metadata catalog? What are the compute engines? That's your architecture diagnosis.

[AI Prompt] Paste this into any AI chatbot: *"For each of these platforms, describe the storage, metadata, and compute layers: Snowflake, Databricks, BigQuery + BigLake, Trino + Iceberg on S3, and Amazon Athena on raw S3 Parquet."*

## TRADE-OFFS — GOVERNANCE, PERFORMANCE, FLEXIBILITY

ChatGPT • AI Assistant



Governance, Performance, Flexibility — The Three-Axis Trade-off

Student

What are the **big trade-offs** across the three architectures?

ChatGPT • AI Assistant

Three axes matter most: **governance** (how well you can control, audit, and secure data), **performance** (how fast queries run at scale), and **flexibility** (what data types

and workloads are supported). No architecture wins on all three.

#### CONCEPT CARD

The trade-off summary:

1. **Governance** - Warehouse (best) > Lakehouse (good) > Lake (weak).
2. **Performance for BI/SQL** - Warehouse (best) ≈ Lakehouse (very close) > Lake (slower).
3. **Flexibility for ML/unstructured** - Lake (best) ≈ Lakehouse (very close) > Warehouse (limited).

Student

When does the lake still win?

ChatGPT • AI Assistant

#### EXAMPLE CARD

The lake is still the right home for:

1. **Raw ingest zones** - everything lands here before curation.
2. **Unstructured data** - images, video, audio for ML training.
3. **Long-term cold storage** - historical data queried rarely.
4. **Cross-cloud portability** - same files readable everywhere.
5. **Pre-processing at massive scale** - Spark jobs over raw JSON logs.

The warehouse (or lakehouse) then serves curated business data downstream.

#### 🔗 ANALOGY

The three axes are like a **triangle where every architecture is a point**. Pull hard toward governance → warehouse. Pull toward flexibility → lake. Try to balance both → lakehouse. Where you land depends on which pull dominates.

#### ⚠️ WARNING

"Flexibility" is not free. Every extra data type in a lake means one more thing to catalog, secure, and clean. Adding governance to a lake is expensive; buying it built-in from a warehouse is easy.

#### ⚠️ COMMON PITFALL / MISTAKE

**Incorrect Way:** Choosing the architecture with the biggest marketing budget instead of matching to your actual workload mix.

**Correct Way:** Inventory your workload - % structured BI, % ML/unstructured, % streaming - then pick the architecture whose strengths match your dominant workload.

#### EXAM TIP CARD

Interview classic: *"When would you pick a warehouse over a lakehouse?"* Winning answer: *"When workload is dominated by BI/SQL on structured data, governance is critical, and the team values operational simplicity over vendor neutrality and cheap storage."*

#### 🔥 EXERCISE

Score your team's use case on a 1-5 scale across the three axes: governance need, structured-vs-unstructured mix, budget sensitivity. The dominant axis usually points at the right architecture.

[AI Prompt] Paste this into any AI chatbot: *"For an early-stage startup (small data, BI-focused), a mid-size fintech (regulated, structured, some ML), and a big-tech consumer app (huge scale, ML-heavy), recommend warehouse, lake, or lakehouse. Justify each."*

## COST DIMENSIONS — STORAGE VS COMPUTE VS QUERY

Student

What actually **costs money** in these systems?

ChatGPT • AI Assistant

Costs break into three buckets: **storage** (bytes at rest), **compute** (CPU/RAM running queries), and **query metadata** (some engines charge per query or per byte scanned). Different architectures spread costs differently - and hidden costs like governance and skills matter too.

#### CONCEPT CARD

Cost structure by architecture:

1. **Warehouse** - moderate-to-high storage cost per TB, compute typically per-warehouse or per-second when active.

2. **Data Lake** - very cheap storage, compute paid to whichever engine you use, plus per-query fees for some tools.
3. **Lakehouse** - very cheap storage (same as lake), compute per-engine, minimal query fees.

Student

What are the sneaky costs?

ChatGPT • AI Assistant

**⚠ WARNING**

Hidden costs bite hard:

1. **Data egress** - moving data OUT of a cloud is expensive; keep compute near storage.
2. **Small-file problem** - lakes and lakehouses need periodic compaction; unmaintained tables get slow AND expensive to query.
3. **Governance tooling** - Immuta, Atlan, Collibra, Unity Catalog all cost real money.
4. **Skill gap** - Spark/Databricks engineers command higher salaries than pure SQL analysts.
5. **Query-per-byte pricing** - accidentally scanning a full table in BigQuery burns thousands of dollars in seconds.

**EXAMPLE CARD**

Rough cost ballpark (2026 approximations, will vary):

1. **S3 storage** - ~\$0.023/GB/month.
2. **Snowflake storage** - ~\$0.04/GB/month (about 2× S3).
3. **BigQuery storage** - ~\$0.02-\$0.023/GB/month for active data.
4. **Snowflake compute** - pay per virtual warehouse per-second, XS costs ~\$2/hr.
5. **BigQuery on-demand** - ~\$5 per TB scanned; slots-based pricing available.
6. **Databricks compute** - VM cost + Databricks Units (DBUs).

**💡 ANALOGY**

Cost structure is like renting an apartment:

- Warehouse = furnished apartment: higher monthly rent, everything included.
- Lake = empty warehouse space: cheap rent, you buy the furniture.

- Lakehouse = empty apartment with utilities included: moderate rent, bring your own furniture but with plumbing and power.

#### ❗ COMMON PITFALL / MISTAKE

**Incorrect Way:** Comparing only storage costs. Compute and egress dwarf storage for active workloads.

**Correct Way:** Model total cost over 12 months across storage + compute + query + egress + governance tools. The winner often changes.

#### EXAM TIP CARD

Interview classic: *"How do you optimize warehouse cost?"* Winning answer: *"Partition and cluster large facts, materialize expensive aggregates, size compute to workload, auto-suspend idle warehouses, monitor scan sizes on per-byte engines, and enforce a policy against SELECT on wide tables."*\*

#### 🔥 EXERCISE

Pick one warehouse or lakehouse platform. Read its pricing page carefully. Compute the monthly cost for: 1 TB of data + 10 hours/day of small compute + 1000 queries/day. Compare with a competitor.

[AI Prompt] Paste this into any AI chatbot: *"Design a cost model for a startup with 5 TB of data landing monthly, 20 analysts running SQL 8 hours a day, and one ML team training weekly. Compare estimated monthly cost on Snowflake, BigQuery, and Databricks in 2026 pricing."*

## OPEN TABLE FORMATS — DELTA, ICEBERG, HUDI

ChatGPT • AI Assistant

```
sql -- Same table, three open formats -- the syntax differences
```

```
-- =====  
-- Delta Lake (Databricks / Spark SQL)  
-- =====  
CREATE TABLE analytics.fact_sales (  
  sales_key    BIGINT,  
  date_key     INT,  
  net_amount   DECIMAL(12, 2)  
)
```

```
USING DELTA
LOCATION 's3://acme-lakehouse/analytics/fact_sales/';

-- Time travel
SELECT * FROM analytics.fact_sales VERSION AS OF 12;

-- =====
-- Apache Iceberg (Snowflake / Trino / BigQuery)
-- =====

CREATE TABLE analytics.fact_sales (
    sales_key    BIGINT,
    date_key     INT,
    net_amount   DECIMAL(12, 2)
)
USING ICEBERG
LOCATION 's3://acme-lakehouse/analytics/fact_sales/';

-- Time travel (Snowflake Iceberg)
SELECT * FROM analytics.fact_sales
    AT (TIMESTAMP ⇒ '2026-07-01 09:00:00');

-- Time travel (Trino Iceberg)
SELECT * FROM analytics.fact_sales
    FOR VERSION AS OF 12345678901234567;    -- snapshot ID

-- =====
-- Apache Hudi (Spark SQL)
-- =====

CREATE TABLE analytics.fact_sales (
    sales_key    BIGINT,
    date_key     INT,
    net_amount   DECIMAL(12, 2)
)
USING HUDI
OPTIONS (
    type = 'cow',                                -- copy-on-write
    primaryKey = 'sales_key'
)
LOCATION 's3://acme-lakehouse/analytics/fact_sales/';

-- All three: same underlying Parquet, different transaction logs.
```

What are **Delta**, **Iceberg**, and **Hudi**?

ChatGPT • AI Assistant

These three are **open table formats** - the metadata layer that turns a folder of Parquet files into a warehouse-class table with ACID transactions, schema evolution, and time travel. They're the technology that makes the lakehouse possible.

#### CONCEPT CARD

The three at a glance:

1. **Delta Lake** - created at Databricks; open-source under Linux Foundation. Simplest to start with. Best documentation and tooling in the Databricks ecosystem.
2. **Apache Iceberg** - created at Netflix; strong open-community governance. Widely adopted by Snowflake, BigQuery, Trino, Dremio. Increasingly the industry default for open interop.
3. **Apache Hudi** - created at Uber. Strong at streaming ingest and incremental updates. Less common but valuable in specific streaming-heavy use cases.

Student

What do they all provide?

ChatGPT • AI Assistant

#### EXAMPLE CARD

Common features across all three:

1. **ACID transactions** - atomic multi-file writes, no partial reads.
2. **Schema evolution** - add / drop / rename columns safely.
3. **Time travel** - query the table as it was at a past point.
4. **UPDATE / DELETE / MERGE** - writes that plain Parquet doesn't allow.
5. **Statistics + data skipping** - smart pruning at query time.
6. **Small-file compaction** - merge many tiny files into fewer big ones.

#### 🔗 ANALOGY

The three formats are like **three brands of file cabinets**. All hold your folders (Parquet files).

All have locks (ACID). All have labels (schema) and a checkout log (transaction history).

Choosing between them is like picking Levi's vs Wrangler - competing brands doing the same job, with differences in ecosystem, community, and features.

#### ⚠️ WARNING

Cross-format interoperability is improving fast - Delta UniForm reads Iceberg metadata, Snowflake supports Iceberg tables natively, and BigQuery reads both. Investigate current state before committing.

#### ❗ COMMON PITFALL / MISTAKE

**Incorrect Way:** Betting on one format for religious reasons. All three are open, all three are converging on the same features, and interop is getting better.

**Correct Way:** Pick based on your primary engine's native support: Databricks-centric → Delta. Snowflake/BigQuery/Trino-centric → Iceberg. Heavy streaming ingest → Hudi.

#### EXAMPLE CARD

Ecosystem support (directional):

1. **Delta** - Native: Databricks. Good support: Trino, Snowflake, DuckDB. Compat mode: many BI tools.
2. **Iceberg** - Native: Snowflake, BigQuery, Trino, Athena, Dremio. Good support: Databricks (via UniForm).
3. **Hudi** - Native: EMR, some Trino. Streaming ingest via Kafka / Flink.

#### EXAM TIP CARD

Interview favorite: *"What does Delta Lake add to Parquet?"* Winning answer: *"A transaction log (usually `_delta_log/`) that provides ACID transactions, schema enforcement, time travel, UPDATE/DELETE/MERGE, and small-file compaction. The Parquet files are unchanged; the log is what makes them behave like a table."*

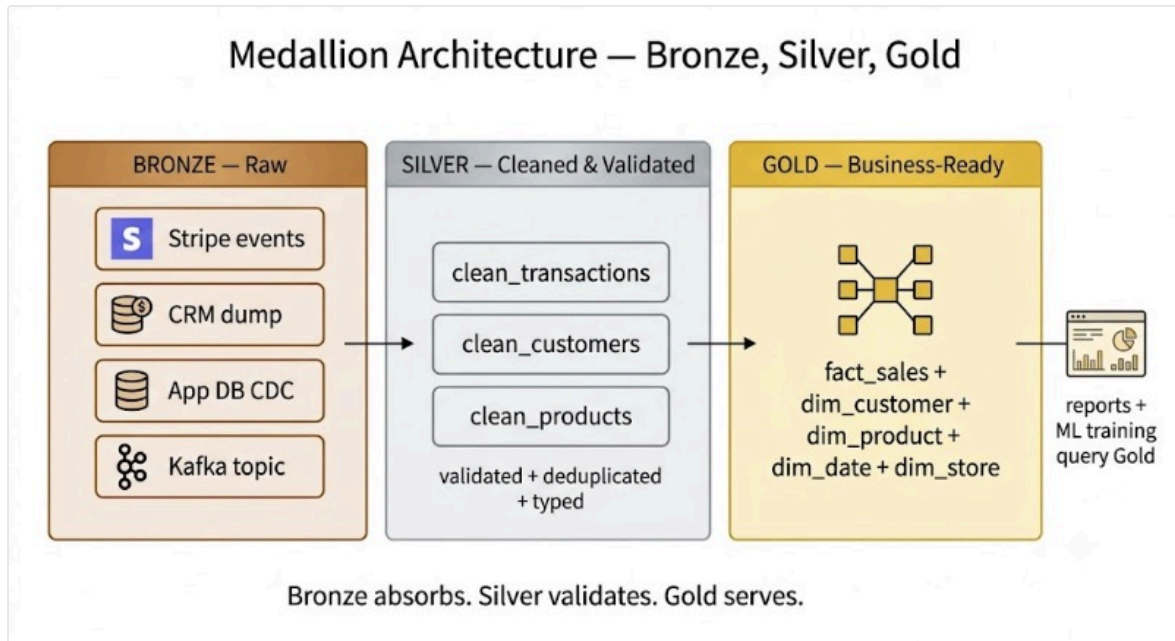
#### 🔥 EXERCISE

For any table format you can access, run a query that goes back in time ( **VERSION AS OF** or **SNAPSHOT\_ID** ). Notice how the same table can return different data at different versions - the time-travel superpower in action.

[AI Prompt] Paste this into any AI chatbot: *"Compare Delta Lake, Apache Iceberg, and Apache Hudi in a table. Include origin, current governance, primary use cases, and one strength each brings uniquely."*

# MEDALLION ARCHITECTURE — BRONZE, SILVER, GOLD

ChatGPT • AI Assistant



Medallion Architecture — Bronze, Silver, Gold

sql — Medallion pipeline in a lakehouse — Bronze → Silver → Gold

```
-- =====
-- BRONZE — raw ingest from source (append-only, untouched)
-- =====

CREATE TABLE bronze.stripe_charges (
  ingest_ts      TIMESTAMP,
  event_json     STRING      -- entire raw payload as JSON
)
USING DELTA
LOCATION 's3://acme-lakehouse/bronze/stripe_charges/';

-- =====
-- SILVER — parsed, validated, deduplicated business entity
-- =====

CREATE TABLE silver.charges (
  charge_id      STRING      NOT NULL,
  customer_id    STRING      NOT NULL,
  amount_usd     DECIMAL(12, 2) NOT NULL,
  currency       VARCHAR(3),
  status         VARCHAR(20),
  charged_ts     TIMESTAMP    NOT NULL,
  ingest_ts      TIMESTAMP
```

```

)
USING DELTA
PARTITIONED BY (DATE(charged_ts))
LOCATION 's3://acme-lakehouse/silver/charges/';

-- Silver transform: parse JSON, dedupe, validate
INSERT INTO silver.charges
SELECT
    event_json:id::STRING                AS charge_id,
    event_json:customer::STRING          AS customer_id,
    (event_json:amount::DOUBLE / 100)::DECIMAL(12, 2) AS amount_usd,
    event_json:currency::STRING          AS currency,
    event_json:status::STRING            AS status,
    event_json:created::TIMESTAMP        AS charged_ts,
    ingest_ts
FROM   bronze.stripe_charges
WHERE  event_json:type::STRING = 'charge.succeeded'
      AND event_json:id IS NOT NULL
QUALIFY ROW_NUMBER() OVER (
    PARTITION BY event_json:id::STRING
    ORDER BY ingest_ts DESC) = 1;

-- =====
-- GOLD — star schema for BI
-- =====

CREATE TABLE gold.fact_charges (
    charge_key      BIGINT,
    date_key        INT,
    customer_key    BIGINT,
    amount_usd      DECIMAL(12, 2),
    is_successful   BOOLEAN
)
USING DELTA
PARTITIONED BY (date_key)
LOCATION 's3://acme-lakehouse/gold/fact_charges/';

-- Reports query GOLD only
SELECT date_key, SUM(amount_usd) AS daily_revenue
FROM   gold.fact_charges
GROUP  BY date_key
ORDER  BY date_key DESC LIMIT 30;

```

What is the **medallion architecture**?

ChatGPT • AI Assistant

The **medallion architecture** is a common lakehouse pattern that organizes data into three **quality tiers**: **Bronze** (raw), **Silver** (cleaned and validated), and **Gold** (business-ready aggregates). Named after Olympic medals to convey progressive refinement.

#### CONCEPT CARD

The three tiers:

1. **Bronze - Raw** - data lands exactly as it arrived from the source. Untouched, unvalidated. Purpose: replay-ability, debugging, audit.
2. **Silver - Cleaned** - Bronze data joined, deduplicated, validated, type-cast. Business rules applied. Still detailed row-level.
3. **Gold - Business-Ready** - star schemas, aggregates, KPIs. This is what BI tools and dashboards query.

Student

Why not just skip to Gold?

ChatGPT • AI Assistant

#### EXAMPLE CARD

Bronze buys you critical properties:

1. **Replay-ability** - if a business rule bug slips into Silver, rerun from Bronze.
2. **Audit trail** - regulators want the raw source form, not just the interpretation.
3. **Late-arriving data** - old records can be added to Bronze and reprocessed forward.
4. **Schema drift buffer** - source system adds a column; Bronze absorbs it, Silver rules decide what to do.
5. **Cheap storage** - Bronze on object storage costs almost nothing.

#### 🗨️ ANALOGY

The medallion pattern is like **food preparation**:

- Bronze = raw vegetables, exactly as delivered from the farm.
- Silver = washed, chopped, cleaned - ready to cook.

- Gold = the plated meal, ready to serve.

You keep the vegetables around so if the chef burns the meal, you can start over from real ingredients.

#### ⚠️ WARNING

Don't build gold tables directly on Bronze. If Bronze has messy raw data, every dashboard becomes a bug hunt. Silver is where the data engineering discipline lives.

#### ❗ COMMON PITFALL / MISTAKE

**Incorrect Way:** Skipping Silver "to save time." Bugs from raw data leak into every gold KPI; fixing them requires redoing everything.

**Correct Way:** Silver is where validation, dedup, joins, and type coercion happen. Gold is where business logic (aggregation, KPIs, star schemas) sits.

#### EXAM TIP CARD

Interview classic: *"What's the medallion architecture?"* Winning answer: *"A three-tier lakehouse pattern: Bronze for raw ingest, Silver for cleaned and validated data, Gold for business-ready analytics. Each tier is a table (typically Delta/Iceberg).*

*Downstream analytics query Gold; audits go back to Bronze."*

#### 🔥 EXERCISE

For any data pipeline you know of, map its stages to Bronze / Silver / Gold. Where does raw source data land? Where does validation happen? Where do star schemas live?

[AI Prompt] Paste this into any AI chatbot: *"Design a medallion architecture for a payment processing company. List the Bronze tables (raw source-shaped), Silver tables (cleaned business entities), and Gold tables (star-schema facts and dims). Show the flow between tiers."*

## CHOOSING FOR YOUR TEAM

Student

How do I **choose** the right architecture for my team?

Use a small decision framework based on three inputs: **data mix** (structured vs unstructured), **team skills** (SQL only vs SQL + Spark), and **workload mix** (BI vs ML vs streaming). The right architecture drops out of those inputs.

#### CONCEPT CARD

The 3-question decision framework:

1. **Is >90% of your workload structured SQL analytics?** → **Warehouse** (Snowflake / BigQuery / Redshift).
2. **Do you have significant unstructured or ML workloads AND a Spark-capable team?** → **Lakehouse** (Databricks / Snowflake + Iceberg / Trino + Iceberg).
3. **Are you doing massive raw ingest and cross-cloud portability?** → **Lake** (plus a downstream warehouse or lakehouse for curated data).

Student

What if I'm just starting?

ChatGPT • AI Assistant

#### EXAMPLE CARD

Starting-point recommendations for 2026:

1. **Small team, structured only** → **Warehouse** (BigQuery for pay-per-query, Snowflake for predictable clusters).
2. **Growing team, mixed BI + some ML** → **Lakehouse** (Databricks or Snowflake with Iceberg tables).
3. **Massive data, heavy ML, multi-cloud** → **Lakehouse on Iceberg** (Trino / Spark + Iceberg on S3/GCS).
4. **Pure batch pipelines, cheap storage first** → **Lake** with a small warehouse for BI on top.

#### 🔗 ANALOGY

Choosing your data architecture is like **choosing a vehicle**:

- Warehouse = a sedan - smooth, reliable, best for daily commutes (BI).
- Lake = a pickup truck - carry anything, but you assemble it yourself.
- Lakehouse = an SUV - comfort of a sedan, capacity of a truck. Modern default.

#### ⚠️ WARNING

Don't over-engineer at the start. If you have 10 GB of data and 3 dashboards, a warehouse is enough - a lakehouse's complexity will slow you down without benefit.

#### ❗ COMMON PITFALL / MISTAKE

**Incorrect Way:** Choosing the trendiest architecture without matching to your workload. A tiny team on "lakehouse" often ends up doing SQL-only work with 5x the operational overhead.

**Correct Way:** Match architecture to workload. Trend-following costs weeks; workload-matching saves them.

#### EXAM TIP CARD

Interview classic: *"How would you decide between a warehouse and a lakehouse for a new team?"* Winning answer: *"Look at workload mix. Pure SQL/BI?*

Warehouse. SQL plus ML plus streaming?

Lakehouse. Cheap raw ingest at massive scale?

Lake with a warehouse or lakehouse on top for curated tables."\*

#### 🔥 EXERCISE

For your organization, fill in these three numbers: (a) % structured SQL workload, (b) % ML/unstructured workload, (c) team's Spark skill level. The dominant number points at the right architecture.

[AI Prompt] Paste this into any AI chatbot: *"Give me a decision tree for choosing between data warehouse, data lake, and lakehouse. Include inputs like data size, workload type, team skills, and budget. Show which architecture wins for 5 typical scenarios."*

## CHAPTER REVIEW & SUMMARY

Student

Can you explain what this chapter is about?

ChatGPT • AI Assistant

### CHAPTER SUMMARY

This chapter zoomed out from schema design to platform choice. You now know where your star, snowflake, and galaxy schemas actually live, and how to reason about the biggest architectural decision your data team will make.

- **DATA WAREHOUSE:** Purpose-built for structured SQL analytics, with tightly coupled storage and compute, strong governance, and best-in-class BI-tool integration. Still the right choice when workload is 95%+ SQL on structured data.
- **DATA LAKE:** Raw files (Parquet, JSON, images) in cheap object storage with schema-on-read. Any data type, huge scale, weak governance - a swamp risk without folder discipline and metastore hygiene.
- **THE LAKEHOUSE:** Lake storage plus an open table format (Delta, Iceberg, Hudi) that adds ACID transactions, schema enforcement, and time travel. The modern industry direction for teams that need both BI and ML on the same physical layer.
- **ARCHITECTURE COMPARISON:** Three layers - storage, metadata, compute. Warehouse tightly couples them; lake loosely couples; lakehouse cleanly separates via an open metadata format.
- **TRADE-OFFS:** Warehouse wins on governance and BI performance; lake wins on flexibility and cost; lakehouse balances all three at the cost of operational complexity.
- **COST DIMENSIONS:** Storage + compute + query + egress + governance tooling + skill costs - model 12 months of total cost, not just storage sticker prices.
- **OPEN TABLE FORMATS - DELTA, ICEBERG, HUDI:** All three add ACID and time travel to Parquet; Delta is Databricks-native, Iceberg is the open multi-engine leader, Hudi excels at streaming ingest.
- **MEDALLION ARCHITECTURE:** Three quality tiers - Bronze (raw), Silver (cleaned), Gold (business-ready) - a discipline that turns a lake or lakehouse into a trustworthy analytics platform.
- **CHOOSING FOR YOUR TEAM:** Match architecture to workload mix, team skills, and data types. Don't chase trends; solve the actual problem.

Student

Can you break down the next concept?

ChatGPT • AI Assistant

You now have the whole picture - from a single fact table row to the multi-cloud lakehouse it lives in. In the next chapter we turn to a completely different world that ML teams inhabit alongside the warehouse: **Feature Store Modeling** - how to store,

serve, and version features for real-time ML predictions without breaking correctness.